

The .claude Folder

A Complete Anatomy

Everything inside your .claude folder and how to actually use it.

by Mark Kashef

Early AI Dopters

This guide accompanies the video walkthrough. Use it as a reference while you set up your own .claude folder. The bash commands on the last page are the fastest way to navigate everything covered here.

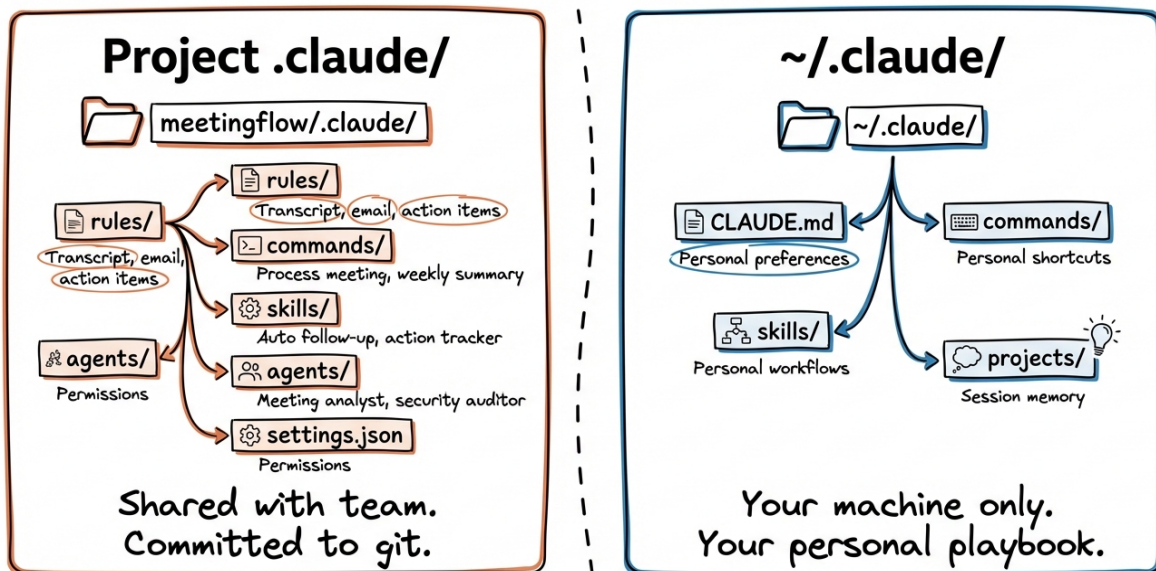
What is in this guide

- 01 The Two .claude Directories
 - 02 CLAUDE.md — The Router
 - 03 Rules — Path-Scoped Intelligence
 - 04 Commands — Your Slash Shortcuts
 - 05 Skills — Always Watching
 - 06 Agents — Built-In Specialists
 - 07 Permissions and Hooks
 - 08 Everything Together — The MeetingFlow Scenario
- Bash Commands Cheat Sheet

The Two .claude Directories

Most people think there is one .claude folder. There are two, and the difference between them is the foundation of everything in this guide.

TWO FOLDERS, ONE BRAIN



Left: project-level .claude folder (inside your repo). Right: global ~/.claude folder (your home directory).

The **project-level .claude folder** lives inside your repository, alongside your source code. Everything in it is scoped to that one project: its rules, its commands, its skills, its agents, its settings. If you are building something sensitive with a large blast radius, this is where you tighten the guardrails. If it is a loose exploratory project, you can open things up. It travels with the repo, which means teammates who clone it get the same setup automatically.

The **home directory ~/.claude folder** is your personal playbook. It contains your preferred timezone, personal shortcuts, tone preferences, and anything else that is specific to you rather than to a project. It also contains your session memory: every conversation Claude Code remembers across sessions lives as a JSON file inside this directory. If you have ever wanted to revisit context from a previous session, it is stored there.

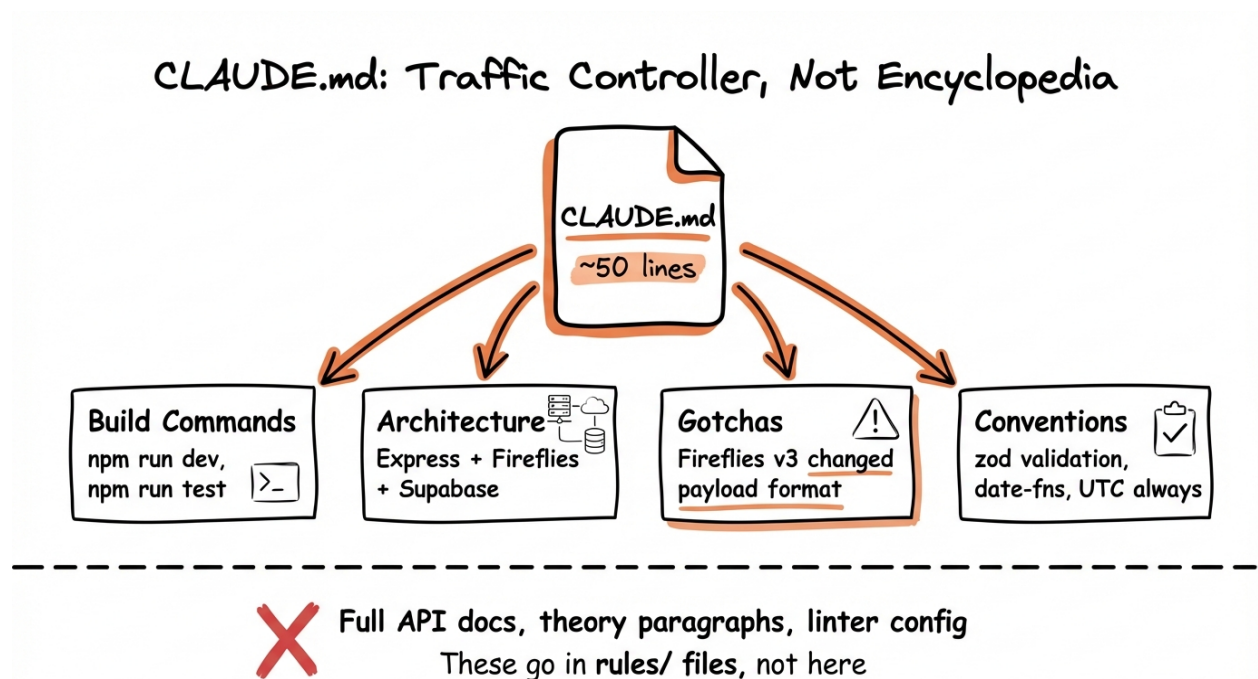
RULE OF THUMB

Share the project-level folder with your team. Keep the home folder to yourself. One is infrastructure. The other is personal configuration.

02

CLAUDE.md — The Router

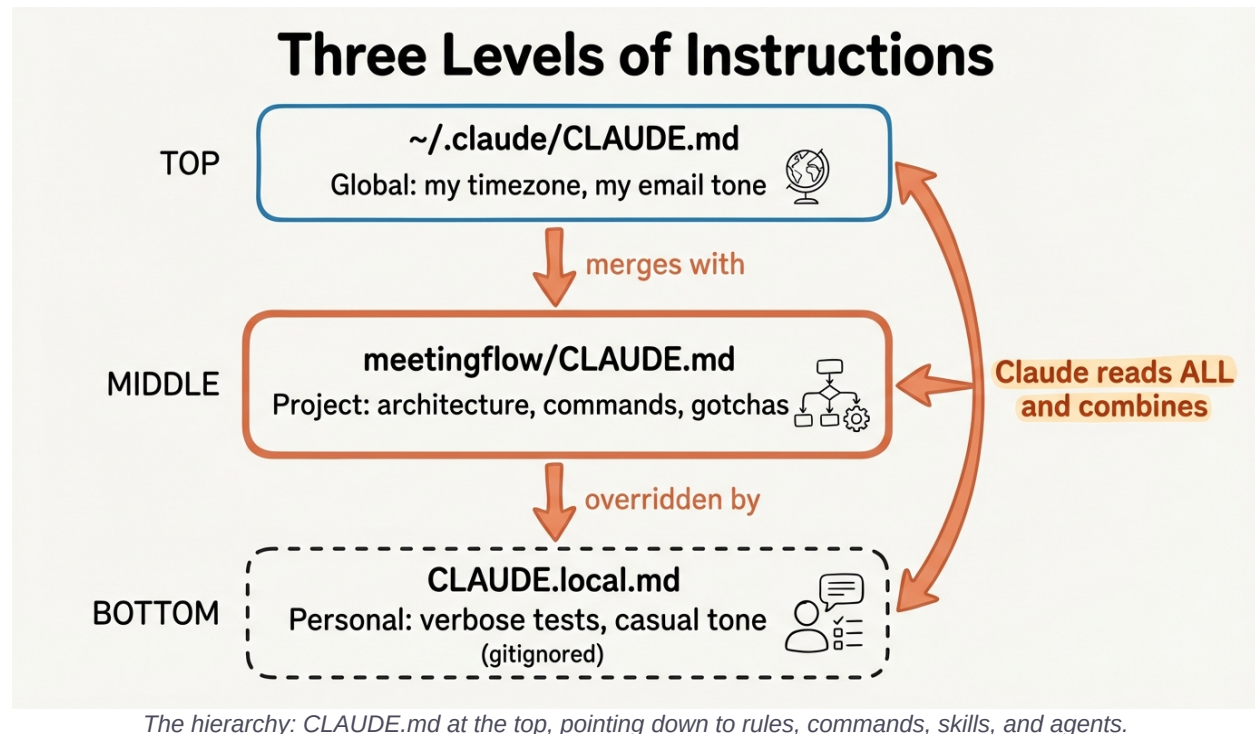
CLAUDE.md is the first file Claude Code reads at the start of every single session. Think of it as air traffic control, not an encyclopedia.



CLAUDE.md acts as the router: it points Claude to the right rules, commands, and conventions rather than holding everything itself.

Because CLAUDE.md is auto-injected into every session, its contents occupy your context window from the first token. This means a bloated CLAUDE.md costs you context on every single request. The goal is to keep it short: 50 lines is a good target. Yours should cover build commands, architecture conventions, any gotchas from the past, and pointers to where deeper rules live. It is more than a memory file and less than a task list.

At the project level, CLAUDE.md is specific to that isolated ecosystem. The MeetingFlow example in this guide has one that covers the Fireflies API, the Supabase connection, Resend for email, and a list of conventions the whole team follows. Any developer (or AI agent) who picks up that repo understands the rules of the road immediately.



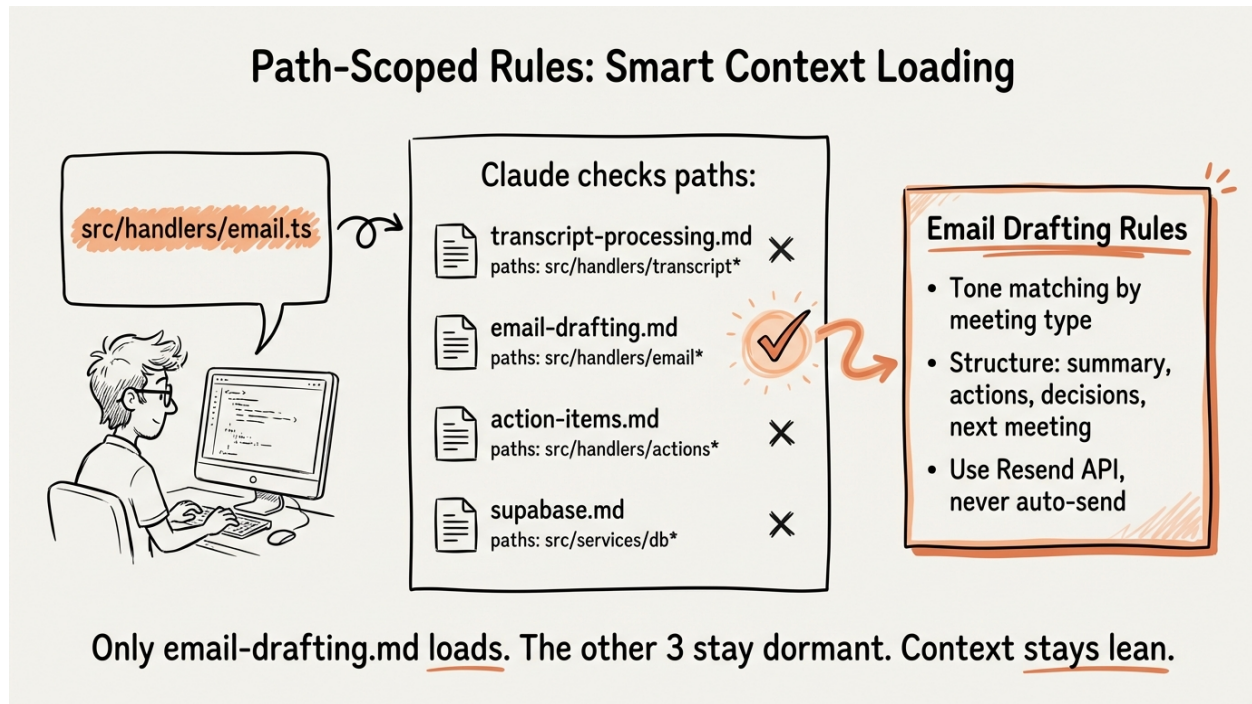
The **CLAUDE.local.md** file is a community convention for personal overrides. Create it in the project root alongside CLAUDE.md, then reference it inside your CLAUDE.md with `@CLAUDE.local.md` so Claude reads both. Since it holds personal preferences that should never end up in version control, you must add it manually to your `.gitignore`. Think of it as an additional user prompt layer: it is where you tell Claude your preferred timezone, your casual email tone, or any personal shortcut that the rest of the team does not need.

QUICK CHECK

Run `!wc -l CLAUDE.md` in Claude Code. If the number is above 80, your CLAUDE.md is probably doing work that belongs in the rules folder instead.

Rules — Path-Scoped Intelligence

Rules are the single most powerful way to keep your CLAUDE.md short while giving Claude deep expertise in specific tasks.

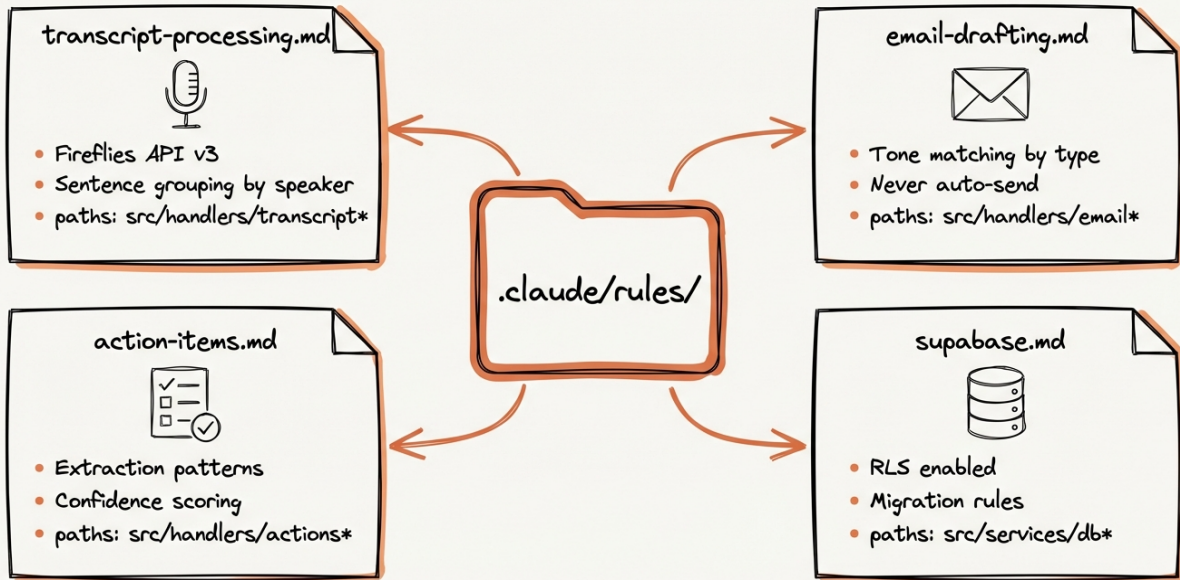


Each rule file has a 'paths:' frontmatter key. Claude only loads a rule file when the active file path matches.

The key insight about rules is that Claude only reads them when they apply. If you are drafting an email, Claude loads the email-drafting rules. If you are writing to the database, it loads the Supabase rules. The rest of your rules stay out of the context window entirely. This means you can have highly detailed, specific rules for ten different workflows without any of them interfering with each other or inflating your costs.

Each rule file starts with frontmatter that tells Claude when to activate it. A `paths:` key lists glob patterns. An `always: true` key loads the rule in every session. A `description:` key alone (with no `paths:`) makes the rule general knowledge that Claude references when relevant. The combination of these triggers is what makes the rules system feel intelligent rather than rigid.

Four Concerns, Four Files, Four Scopes



The rules ecosystem for MeetingFlow: four rule files, each scoped to a different part of the workflow.

In the MeetingFlow example, the rules folder contains four files: `transcript-processing.md` (Fireflies API conventions), `email-drafting.md` (tone matching and Resend API), `action-items.md` (extraction format and confidence scoring), and `supabase.md` (connection patterns and RLS rules). None of these are in `CLAUDE.md`. `CLAUDE.md` simply points to the rules folder and says: when you are doing email work, go and read the email rules. The actual detail lives in the rule file and loads only when needed.

SELF-IMPROVING RULES

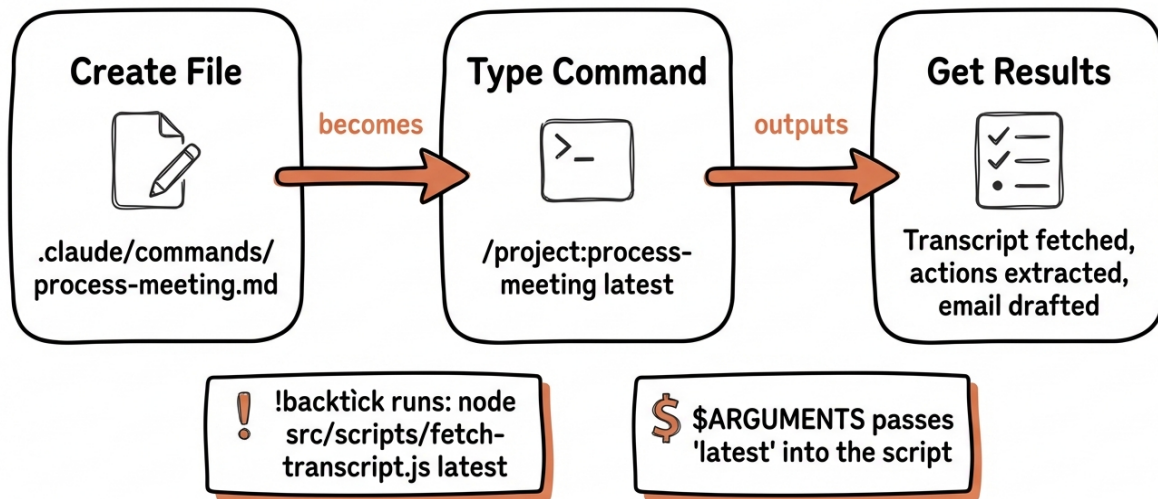
Rules are not static. As you use them and find gaps, you update the file. After a few weeks of active use, a good rule file becomes extremely precise because it has been refined against real situations rather than hypothetical ones.

04

Commands — Your Slash Shortcuts

Commands are the simplest component of the `.claude` folder to understand. The rule is direct: the filename is the slash command.

From File to Slash Command in Seconds



A command file in `.claude/commands/` becomes a `/project:` slash command. The filename is the command name.

If you have a file at `.claude/commands/process-meeting.md`, you invoke it by typing `/process-meeting` in Claude Code. The file contains a natural language description of what the command should do, any structure or step-by-step instructions, and optionally a `$ARGUMENTS` placeholder. That placeholder is the wild card: it lets you pass a sub-argument after the slash command to increase specificity. In the MeetingFlow example, `/process-meeting latest` pulls the most recent Fireflies transcript, while `/process-meeting [meeting-id]` jumps directly to a specific recording.

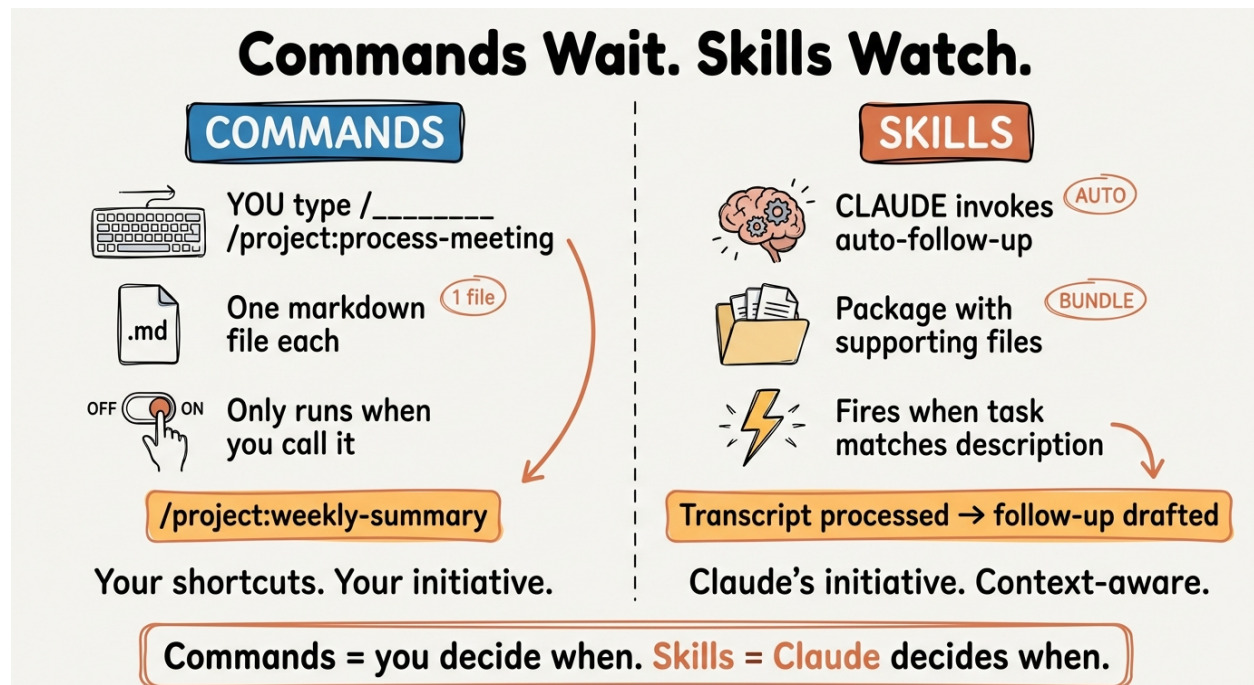
Commands are things that wait to be invoked. They do nothing until you type the slash command. This is the key difference between commands and skills, which are covered in the next section. A command is a deliberate, manual trigger. You decide when it runs and you can pass context to it via the argument.

QUICK LIST

To see every command available in your project, run: `!!s .claude/commands/` Each filename maps directly to a slash command you can use right now.

Skills — Always Watching

Skills are the component that separates an advanced .claude folder from a basic one. Unlike commands, skills are not waiting to be called. They are watching.



Commands wait. Skills watch. The trigger is in the skill's description field, not in your message.

A skill fires automatically when Claude recognizes trigger words in its description field during a session. You do not call it directly. Instead, as Claude is working through a task, it reads the descriptions of all active skills, notices a match, and invokes the skill without you doing anything. In the MeetingFlow example, the auto-follow-up skill watches for any session that involves processing a meeting transcript. The moment it detects that context, it drafts the follow-up emails automatically.

Skills are also composable. Inside a skill file, you can reference rule files using the @ syntax, which means a skill can load a specific rule the moment it fires. The action-tracker skill in MeetingFlow loads the action-items rule as part of its execution. One trigger cascades into multiple layers of instructions, all loaded at exactly the right moment.

EXPERT TIP — disable-model-invocation

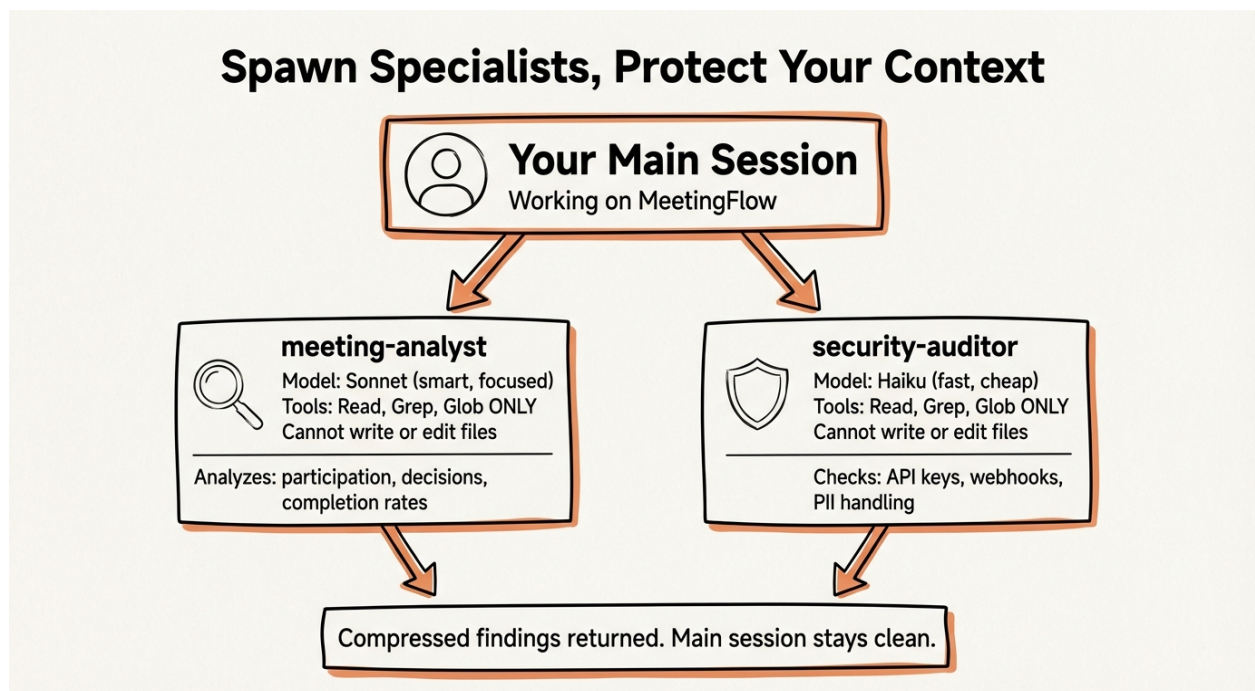
If you have a skill that should only run when you explicitly call it, add `disable-model-invocation: true` to the skill's frontmatter. This is useful when you have multiple similar skills and you want precise control over which one runs in a specific edge case. You can also add `context: fork` to run the skill in its own context window, separate from your main session.

The `allowed-tools` field in a skill's frontmatter restricts which tools the skill can use. A skill that only reads transcripts has no reason to write files or run bash commands. Locking it to `Read` and `Grep` keeps it focused and prevents scope creep during execution.

06

Agents — Built-In Specialists

Agents are pre-defined sub-agents you have crystallized because you invoke them regularly in the same way. Think of them as named specialists you hire once.



Two agents in MeetingFlow: meeting-analyst (Sonnet) for transcript review and security-auditor (Haiku) for permission checks.

The defining feature of an agent definition file is the model field. You specify exactly which model runs this agent every time. In MeetingFlow, the meeting-analyst uses Claude Sonnet because transcript analysis requires reasoning but not maximum power. The security-auditor uses Claude Haiku because it is running targeted, narrow checks where speed and cost matter more than depth. You control the cost-to-capability trade-off per agent.

The tools field restricts what the agent can do. Both agents in MeetingFlow are limited to Read, Grep, and Glob. They can look at files but they cannot write, run commands, or touch the database. This containment is intentional. An agent that only needs to read transcripts and flag issues should not have write access. The tools field enforces that boundary at the agent level rather than relying on you to remember to constrain it each time.

STRUCTURING AGENT INSTRUCTIONS

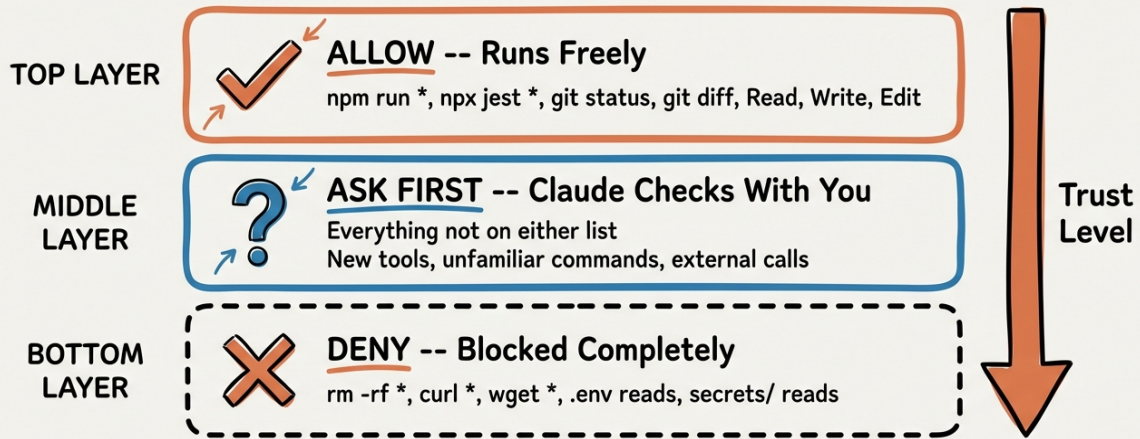
End every agent definition with a concrete output format. For example: 'Report findings with severity: critical / high / medium / low and include specific file and line references.' Vague instructions produce vague output. Precise output format instructions produce reports you can act on immediately.

07

Permissions and Hooks

settings.json is where you define what Claude Code is allowed to do, what it must ask about first, and what it must never touch.

THE PERMISSION SANDWICH



The middle ground is the safety net. You don't have to predict every command.

Three tiers: allow (runs freely), ask (human-in-the-loop), deny (blocked entirely).

The permissions block inside `settings.json` has two arrays: `allow` and `deny`. Anything in `allow` runs without asking you. Anything in `deny` is blocked outright, even if Claude thinks it would be helpful. The everything-else behavior depends on your current mode: in `bypass` mode it runs freely, in `accept-edits` mode it asks for confirmation. The new `auto` mode sits between these two extremes and makes increasingly better decisions about what needs a human in the loop as the model improves.

If you use Claude Code in `accept-edits` mode, it writes down each permission you grant into your `settings.json` automatically. Over time, this file becomes a precise record of exactly what you trust Claude to do autonomously in this project. You can then share that file with a teammate as a starting point for their own setup.

Hooks live inside `settings.json`, not in a separate folder. There is no `hooks/` directory. The `hooks` object inside `settings.json` has lifecycle events: `PreToolUse` fires before Claude runs a tool, `PostToolUse` fires after. Each hook specifies a matcher (which tool to intercept) and a shell command to run. In `MeetingFlow`, a `PreToolUse` hook blocks any write to `.env` files by exiting with code 2. A `PostToolUse` hook logs every bash command to `.claude/hook.log` for audit purposes.

ALL HOOK EVENTS

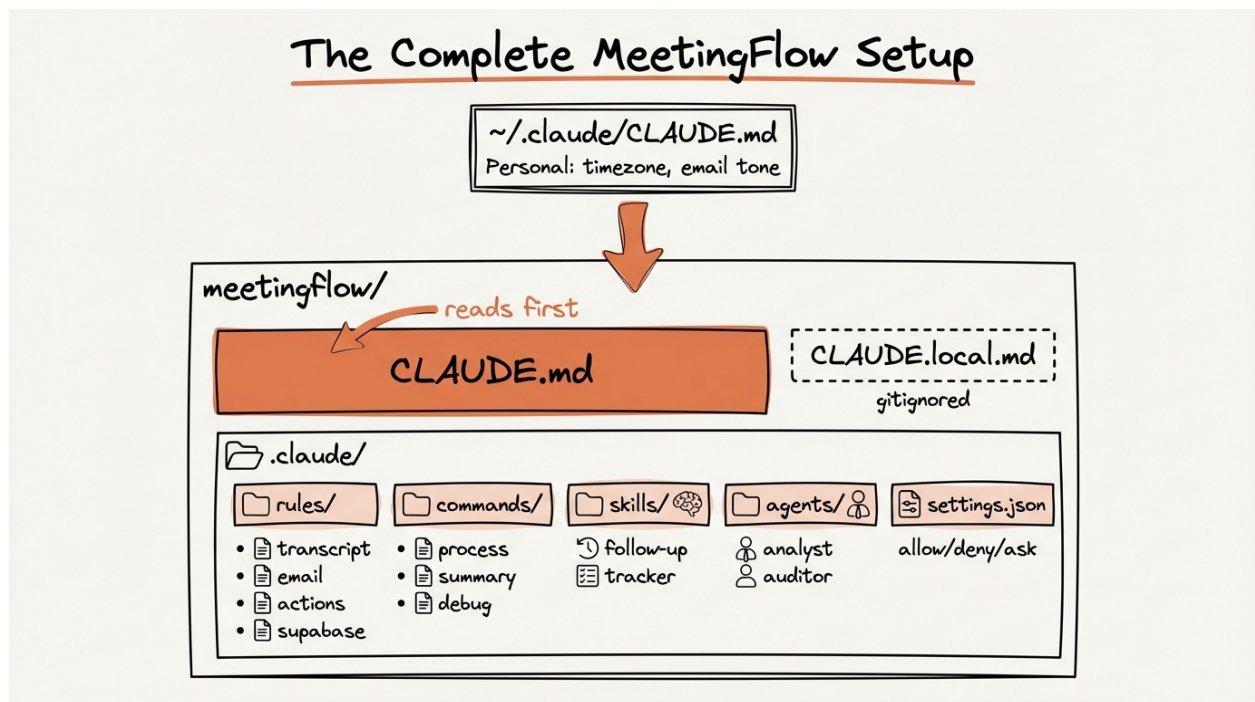
To get the full list of hook events available, type: @claude-code-guide list every single hook possible and when to use them. You will get all available hook lifecycle events with descriptions of when each fires.

The settings.local.json file follows the same pattern as CLAUDE.local.md. It is your personal permission overrides on top of the shared team settings. Maybe you have granted yourself access to a tool that a less experienced team member should still be prompted about. Local settings handle that without changing what everyone else sees.

08

Everything Together

Here is how all seven components work as a single system in the MeetingFlow project.

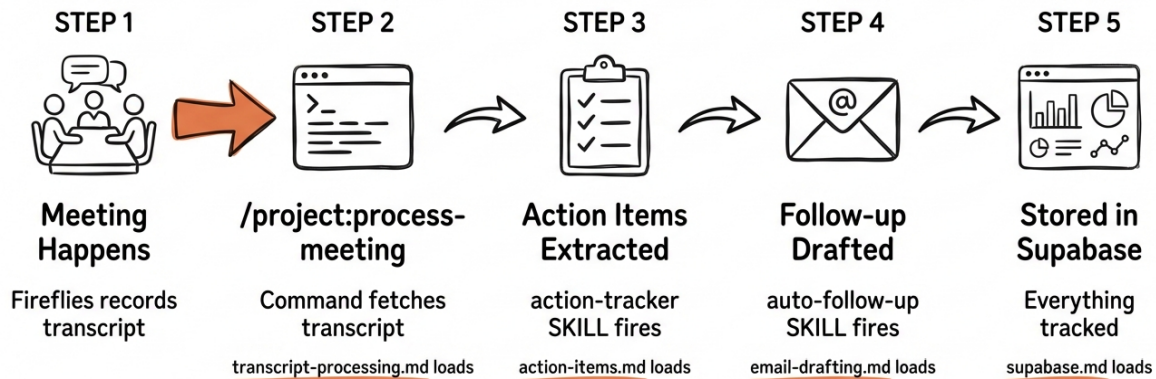


The complete MeetingFlow .claude folder: all seven components in their positions.

MeetingFlow is a hypothetical Node.js project that pulls meeting transcripts from the Fireflies API, extracts action items, drafts follow-up emails via Resend, and stores everything in Supabase. The

.claude folder for this project has one file for each component discussed in this guide.

From Meeting to Follow-up in One Command



One command. Four rules loaded contextually. Two skills fired automatically.

End-to-end flow: slash command triggers transcript processing, skills fire automatically, rules load on demand, output lands in Supabase.

Step one: a meeting ends and Fireflies records the transcript. Step two: you run `/process-meeting latest` inside Claude Code. This command uses the `$ARGUMENTS` placeholder to pull the most recent transcript from the Fireflies API. As Claude processes it, the `transcript-processing.md` rule loads automatically because the active context matches its path scope.

Step three: Claude detects action items in the transcript. This triggers the action-tracker skill, which loads the `action-items.md` rule for its extraction format and confidence-scoring instructions. Simultaneously, the auto-follow-up skill fires because the trigger words in its description (next steps, follow-up, action items) match the context. It drafts the emails using the `email-drafting.md` rule for tone and the Resend API details.

Step four: the meeting-analyst agent reviews the full output and the security-auditor agent checks that no sensitive data was written anywhere it should not be. Both run on the models you specified in their definitions. Both are locked to read-only tools. Step five: everything lands in Supabase under the `supabase.md` connection rules. The `settings.json` hooks have been logging every bash command throughout. The `PreToolUse` hook ensured nothing touched the `.env` file.

The entry point for all of this was a single slash command and the word 'latest.' The rest cascaded automatically from the `.claude` folder structure you built once and now reuse for every meeting.

Bash Commands Cheat Sheet

Run these with an exclamation mark prefix inside Claude Code (e.g. `!ls .claude/`) or directly in your terminal. No API credits used.

See everything in your project `.claude` folder

Shows the top-level contents: agents, commands, rules, skills, settings.json.

```
ls .claude/
```

See everything in your global `~/.claude` folder

Your personal playbook: global rules, session memory, personal shortcuts.

```
ls ~/.claude/
```

Read your project `CLAUDE.md`

Prints the entire file. Fast way to review what Claude reads at session start.

```
cat CLAUDE.md
```

Check how long your `CLAUDE.md` is

Line count. Target: under 80. Over that, move content into rules files.

```
wc -l CLAUDE.md
```

Preview the first 6 lines of a rule

Quick sanity check on frontmatter and opening instructions.

```
head -6 .claude/rules/email-drafting.md
```

Preview the first 15 lines of a rule

Enough to see the scope and key instructions without printing the full file.

```
head -15 .claude/rules/email-drafting.md
```

Read a full command file

Shows the command description, structure, and any \$ARGUMENTS placeholder.

```
cat .claude/commands/process-meeting.md
```

List all commands in the project

Every filename here is a slash command you can invoke right now.

```
ls .claude/commands/
```

Read a skill file

Shows the trigger description and allowed-tools restrictions.

```
cat .claude/skills/auto-follow-up/SKILL.md
```

Read two agent files at once

Pass multiple files to cat to print them in sequence.

```
cat .claude/agents/meeting-analyst.md  
.claude/agents/security-auditor.md
```

Review your settings and hooks

Shows all allow/deny permissions and any hooks you have defined.

```
cat .claude/settings.json
```

Compare project vs global .claude at a glance

Run both in one line to see project-level vs personal config side by side.

```
ls .claude/ && ls ~/.claude/
```

For more Claude Code guides, deep dives, and a community of builders:

Early AI Dopters